

# REC 5 Ontology Translation Principles and Methodology

## Overview

This document outlines the semantic and ontological principles used in translating REC 5 (Real Estate Core version 5) from DTDL (Digital Twin Definition Language) to OWL/SHACL. The approach establishes a hybrid semantic model that combines OWL reasoning capabilities with SHACL validation constraints.

**REC 5 Evolution:** This version represents a significant architectural shift where REC focuses purely on real estate entities while ICT equipment and advanced telemetry capabilities have been migrated to Brick Schema for better semantic separation and specialized domain coverage.

## Foundational Semantic Principles

### 1. Universal Entity Architecture

**Principle:** All REC concepts inherit from a single root entity class.

```
rec:Entity a owl:Class ;  
  rdfs:label "Entity"@en, "Entitet"@sv ;  
  rdfs:comment "All REC classes will be subclasses of the rec:Entity."@en .
```

**Rationale:** Provides unified semantics and shared properties across all REC concepts.

### 2. Universal Relationship Hierarchy

**Principle:** All object properties inherit from a single root relationship property.

```
rec:relationship a owl:ObjectProperty ;  
  rdfs:label "relationship"@en, "relation"@sv ;  
  rdfs:comment "All REC relationships will be sub properties of  
rec:relationship."@en .
```

**Benefits:**

- Enables relationship-agnostic queries
- Provides extension points for cross-cutting concerns
- Supports generic relationship reasoning

### 3. Dual OWL/SHACL Modeling

**Principle:** Every class is simultaneously an OWL Class and a SHACL NodeShape.

```
rec:Space a owl:Class, sh:NodeShape ;  
  rdfs:label "Space"@en, "Utrymme"@sv ;  
  rdfs:subClassOf rec:Entity ;  
  sh:property rec:geometryShape, rec:georeferenceShape .
```

**Advantages:**

- **OWL:** Enables reasoning, inference, and semantic queries
- **SHACL:** Provides structural validation and constraint checking
- **Integration:** Single model serves both purposes

#### 4. Property Shape Pattern

**Principle:** Each object property has a corresponding SHACL PropertyShape with validation constraints.

```
# Object Property
rec:geometry a owl:ObjectProperty ;
  rdfs:label "geometry"@en, "geometri"@sv ;
  rdfs:subPropertyOf rec:relationship .

# Corresponding SHACL Shape
rec:geometryShape a sh:PropertyShape ;
  rdfs:label "geometry shape"@en, "geometri form"@sv ;
  sh:path rec:geometry ;
  sh:class rec:Geometry ;
  sh:maxCount 1 .
```

**Structure:**

- **Path:** Links to the property being constrained
- **Class:** Specifies target class for object properties
- **Cardinality:** Defines min/max occurrence constraints

#### 5. Multilingual Documentation

**Principle:** All entities include English and Swedish labels.

```
rec:Georeference a owl:Class, sh:NodeShape ;
  rdfs:label "Geo reference"@en, "Georeferens"@sv ;
  rdfs:subClassOf rec:Entity .
```

**Standard Translations:**

- Entity → Entitet
- Space → Utrymme
- Geometry → Geometri
- Georeference → Georeferens
- Relationship → Relation
- Property → Egenskap
- Asset → Tillgång
- Collection → Samling

#### 6. Inverse Relationship Modeling

**Principle:** Bidirectional relationships are explicitly modeled with inverse properties.

```
rec:hasPart a owl:ObjectProperty ;
  rdfs:label "has part"@en, "har del"@sv ;
  rdfs:subPropertyOf rec:relationship .
```

```
rec:isPartOf a owl:ObjectProperty ;
  rdfs:label "is part of"@en, "är del av"@sv ;
  rdfs:subPropertyOf rec:relationship ;
  owl:inverseOf rec:hasPart .
```

## 7. Cardinality Constraints

**Principle:** Use SHACL constraints rather than OWL restrictions for validation.

```
rec:isPartOfShape a sh:PropertyShape ;
  rdfs:label "is part of shape"@en, "är del av form"@sv ;
  sh:path rec:isPartOf ;
  sh:class rec:Space ;
  sh:maxCount 1 . # Maximum one parent space
```

**Rationale:** SHACL provides better validation semantics than OWL restrictions.

## Work Process and Methodology

### Phase 1: Analysis and Separation

1. **DTMI Analysis:** Identify REC vs Brick content by namespace patterns
  - dtmi:org:w3id:rec:\* → REC content (269 files)
  - dtmi:org:brickschema:schema:Brick:\* → Brick content (1,094 files)
2. **Translation Scope:** Focus on REC-only content, reference existing Brick ontology
3. **Challenge Identification:** Document translation challenges
  - Components → Separate classes
  - Enumerations → SKOS/OWL choices
  - Maps → Intermediate classes
  - Brick integration → Property linking

### Phase 2: Semantic Design

1. **Establish Foundations:** Create universal base classes and properties
2. **Design Patterns:** Define consistent modeling approaches
3. **Multilingual Support:** Add Swedish translations for accessibility
4. **Validation Strategy:** Combine OWL semantics with SHACL validation

### Phase 3: Implementation Architecture

```
src/
├─ translators/           # Modular translation components
│  ├─ InterfaceTranslator.js # DTDL Interface → OWL Class + SHACL Shape
│  ├─ PropertyTranslator.js  # Properties → Data/Object Properties
│  ├─ RelationshipTranslator.js # Relationships → Object Properties + Shapes
│  └─ EnumerationTranslator.js # Enums → SKOS/OWL strategies
├─ mappings/             # Configuration-driven translations
│  ├─ namespace-mappings.json # DTMI → IRI mappings
│  └─ label-translations.json  # English/Swedish label pairs
```

```
| └─ datatype-mappings.json    # DTDL → XSD type mappings
└─ RecTranslator.js           # Main orchestrator
```

## Phase 4: Quality Assurance

1. **Validation:** Ensure generated SHACL shapes validate correctly
2. **Reasoning:** Test OWL reasoning and inference capabilities
3. **Integration:** Verify REC-Brick linking through `rec:hasPoint`
4. **Documentation:** Maintain multilingual labels and comments

## Translation Rules

### Core Exclusion Rules (Following Recommendations)

#### ✗ Excluded from Translation:

- `customProperties` - Use external SHACL extensions instead
- `customTags` - Use external SHACL extensions instead
- DTDL semantic types ( `["Property", "Temperature"]` ) - Redundant with Brick Point context
- Observation subclasses - Use single `rec:ObservationEvent` with Brick Point semantics
- `Architecture.isFedBy` substance enum - Use Brick substance classes instead

#### ✓ Included in Translation:

- `name` - Core semantic property for all entities
- `identifiers` - Critical for data integration via intermediate classes
- Components - Translated to separate OWL classes with relationships
- Small enumerations (2-10 values) - SHACL `sh:in` constraints only (no separate OWL classes)

### Enumeration Consistency Principle

**Critical Rule:** All enumerations must use SHACL `sh:in` constraints as datatype properties. Never create separate OWL enumeration classes that break universal inheritance from `rec:Entity`.

```
# ✓ Correct approach – SHACL constraint maintains hierarchy
rec:ServiceObjectShape sh:property [
  sh:path rec:severity ;
  sh:in ( "Major" "Minor" "Severe" ) ;
  sh:datatype xsd:string
] .

# ✗ Wrong approach – Creates orphan class outside rec:Entity hierarchy
rec:SeverityType a owl:Class . # NOT a subclass of rec:Entity!
rec:SeverityType_Major a rec:SeverityType . # Breaks foundational pattern
```

**Rationale:** Discovered during Protege review that `owl_individuals` strategy creates standalone classes that violate the universal `rec:Entity` inheritance principle. All REC concepts must inherit from `rec:Entity`.

## Configuration Decision Record

### Decision 1: Validation Messages Strategy (DECIDED)

**Choice:** Basic English validation messages with future multilingual support

**Options Considered:**

- A) Minimal approach (no validation messages)
- B) Basic validation messages in English only ☒ **SELECTED**
- C) Full multilingual messages (English + Swedish)

**Rationale:**

- **Developer Experience:** English messages provide immediate clarity for validation failures
- **Maintainability:** Single language reduces translation burden initially
- **Future-Ready:** Architecture supports adding Swedish messages later
- **Brick Alignment:** Matches Brick's validation message approach

**Implementation:**

```
# Add to all PropertyShapes
rec:geometryShape sh:message "Geometry must be a valid rec:Geometry instance"@en .
rec:hasPointShape sh:message "Assets must reference valid brick:Point instances"@en
.
```

**Future Enhancement:** Extend to multilingual support by adding Swedish ( `@sv` ) messages alongside English ones.

## Decision 2: Property Characteristics Strategy (DECIDED)

**Choice:** Add essential characteristics (asymmetric, irreflexive) ☒ **SELECTED**

**Options Considered:**

- A) Keep current basic properties
- B) Add essential characteristics (asymmetric, irreflexive) ☒ **SELECTED**
- C) Add full semantic properties (transitive, functional)

**Rationale:**

- **Safety First:** Prevents logical inconsistencies in spatial/hierarchical relationships
- **Practical Reasoning:** Enables useful consistency checking without overwhelming complexity
- **Brick Alignment:** Follows Brick's pattern of careful characteristic selection for critical properties
- **Future Ready:** Can expand to full semantic properties later if needed

**Implementation:**

```
# Essential characteristics for spatial and hierarchical properties
rec:hasPart a owl:ObjectProperty, owl:AsymmetricProperty, owl:IrreflexiveProperty ;
  rdfs:comment "Prevents circular part-of relationships" .

rec:isPartOf a owl:ObjectProperty, owl:AsymmetricProperty, owl:IrreflexiveProperty ;
  rdfs:comment "Prevents circular containment hierarchies" .

rec:locatedIn a owl:ObjectProperty, owl:AsymmetricProperty, owl:IrreflexiveProperty
;
  rdfs:comment "Prevents spatial containment loops" .
```

```
rec:hasPoint a owl:ObjectProperty, owl:AsymmetricProperty, owl:IrreflexiveProperty ;  
  rdfs:comment "Ensures clean asset-to-point relationships" .
```

**Brick Schema Alignment:** This matches Brick's approach where critical relationships like `brick:feeds` use `owl:AsymmetricProperty`, `owl:IrreflexiveProperty` to prevent logical inconsistencies while keeping complexity manageable.

### Decision 3: Domain Strategy (DECIDED)

**Choice:** Minimal domains (SHACL-first approach)  **SELECTED**

#### Options Considered:

- A) Minimal domains (current SHACL-first)  **SELECTED**
- B) Comprehensive domains (OWL-first)
- C) Hybrid approach (signature properties only)

#### Rationale:

- **SHACL-First Consistency:** Aligns with our established pattern of using SHACL shapes for targeting
- **Universal Querying:** Properties remain universally applicable, enabling flexible queries across `rec:Entity` hierarchy
- **Brick Schema Alignment:** Matches Brick's minimal domain approach with extensive SHACL constraints
- **Semantic Flexibility:** Avoids over-constraining properties that may apply to multiple entity types

#### Implementation:

```
# Minimal domains – only on signature properties that define class identity  
rec:hasPoint a owl:ObjectProperty ;  
  rdfs:domain rec:Asset ; # Assets are entities that can have measurement points  
  rdfs:range brick:Point .  
  
# Most properties have no domain – targeting via SHACL shapes  
rec:geometry a owl:ObjectProperty ;  
  rdfs:subPropertyOf rec:relationship ; # No domain restriction  
  # Targeting: sh:targetClass in relevant NodeShapes  
  
rec:hasPart a owl:ObjectProperty ;  
  rdfs:subPropertyOf rec:relationship ; # Universal part-of relationship  
  # Targeting: Applied to specific classes via SHACL shapes
```

#### Benefits:

- Enables queries like `?entity rec:relationship ?target` across all REC entities
- Properties can be applied to new entity types without ontology changes
- Consistent with Brick Schema integration patterns
- Maintains semantic clarity through SHACL shape constraints

### Decision 4: Missing Range Strategy (DECIDED)

**Choice:** Hybrid approach – constrained ranges for business-specific relationships, open ranges for universal concepts  **SELECTED**

## Systematic Analysis Results:

### Constrained Ranges (Business-specific relationships):

- `Portfolio.includes` → `rec:Architecture` | `rec:Collection` (buildings, sites, apartments, campuses)
- `Lease.leaseOf` → `rec:Collection` | `rec:Space` (premises/apartments or direct spaces)
- `Space.isLocationOf` → `rec:Asset` | `rec:BuildingElement` (inverse consistency with `locatedIn`)
- `Document.url` → `xsd:anyURI` as `dataProperty` (correct semantic modeling)

### Open Ranges (Universal concepts):

- `Agent.owns` - Ownership is universal, supports external ontologies
- `Document.documentTopic` - Documents can describe anything
- `PointOfInterest.objectOfInterest` - POIs can reference anything interesting
- `ServiceObject.relatedTo` - Service management spans all domains

### Rationale:

- **Business Semantics:** Real estate and spatial relationships have clear domain boundaries
- **Universal Concepts:** Ownership, documentation, and service relationships are cross-domain
- **Inverse Consistency:** Spatial relationships maintain logical coherence with their inverses
- **Extensibility:** Open ranges support integration with Brick, QUDT, and other ontologies

### Implementation Pattern:

```
# Constrained for business clarity
rec:includes rdfs:range [ owl:unionOf (rec:Architecture rec:Collection) ] .

# Open for universal applicability
rec:owns a owl:ObjectProperty ;
  rdfs:subPropertyOf rec:relationship .
  # No range constraint – universally applicable
```

## Decision 5: Enumeration Strategy Refinement (DECIDED)

**Choice:** Keep uniform SHACL constraint strategy  **SELECTED**

### Options Considered:

- A) Keep uniform SHACL constraints for all enumerations  **SELECTED**
- B) Allow mixed strategies (some SKOS, some OWL individuals per enumeration)

### Rationale:

- **Consistency:** All enumerations follow the same pattern
- **Proven Approach:** Successfully resolved enumeration hierarchy issues
- **Simplicity:** No complex strategy selection logic needed in translator
- **Effective Validation:** SHACL constraints provide clear validation semantics

**Implementation:** All enumerations continue to use `sh:in` constraints as datatype properties, maintaining the universal `rec:Entity` inheritance principle.

## Decision 6: Component Mapping and Observation Strategy (DECIDED)

**Choice:** Complete component mapping with observation event exclusion  **SELECTED**

#### Component Mapping Analysis:

##### REC Components (Mapped):

- `identifiers` → Universal `rec:hasIdentifier` → `rec:Identifier` mapping
- `Architecture.area` → `rec:hasArea` → `rec:ArchitectureArea`
- `Architecture.capacity` → `rec:hasCapacity` → `rec:ArchitectureCapacity`

##### Brick Components (Excluded):

- `lastKnownValue` → Use existing `brick:lastKnownValue` property instead
- All other Brick Point components → Outside REC scope

**Critical Discovery:** Brick Schema already defines comprehensive observation handling:

```
# Existing Brick infrastructure
brick:lastKnownValue rdf:type brick:EntityProperty ;
    rdfs:domain brick:Point ;
    rdfs:range bsh:LastKnownValueShape .

# Brick's value structure (simple timestamp + value, NOT events)
bsh:LastKnownValueShape
    sh:property [sh:path brick:timestamp ; sh:datatype xsd:dateTime] ;
    sh:property [sh:path brick:value] .
```

**Observation Strategy Decision:** Exclude `rec:ObservationEvent` entirely

##### Rationale:

- **Historical Legacy:** `rec:ObservationEvent` is a remnant from REC 3.x when REC had its own capabilities model
- **Architectural Clarity:** REC 5 + Brick integration pattern is:
  - **REC:** Real estate entities (spaces, assets, agents, equipment)
  - **Brick:** Measurement points, ICT equipment, and telemetry infrastructure
  - **Integration:** REC entities connect to Brick Points via `rec:hasPoint`
  - **ICT Migration:** ICTEquipment and ICTHardware are now part of Brick Schema
- **No Duplication:** Brick's observation infrastructure is complete and proven
- **Simplified Model:** Eliminates unnecessary complexity from dual observation systems

##### Implementation Pattern:

```
# REC provides real estate context
ex:HVAC_System_01 a rec:Equipment ;
    rec:hasPoint ex:TempSensor_01 .

# Brick provides measurement capability
ex:TempSensor_01 a brick:Temperature_Sensor ;
    brick:lastKnownValue [
        brick:value 22.5 ;
        brick:timestamp "2024-01-15T10:30:00Z"^^xsd:dateTime
    ] .
```

**Component Mapping Conclusion:** REC component mappings are complete with 3 patterns (universal identifiers + Architecture area/capacity). No additional mappings needed – Brick handles telemetry components.

**REC 5 ICT Equipment Migration:** The entire ICTEquipment and ICTHardware hierarchy has been migrated from REC to Brick Schema to consolidate all ICT/networking equipment under Brick's comprehensive technical infrastructure coverage. This includes:

**Excluded ICT Equipment Classes:**

- ICTEquipment (root class) and ICTHardware (hardware root)
- DataNetworkEquipment and all networking subclasses (EthernetPort, EthernetSwitch, NetworkRouter, NetworkSecurityEquipment, WirelessAccessPoint)
- Controllers and communication equipment (Controller, BACnetController, ModbusController, Gateway)
- Server and infrastructure equipment (Server, ITRack)
- Sensor equipment controllers (SensorEquipment and all sensor subclasses like DaylightSensorEquipment, IAQSensorEquipment, etc.)
- AudioVisualEquipment and connectivity parameters (GatewayConnectionParameter, IoTHubConnectionParameter)

This comprehensive migration ensures:

- **Clean Domain Separation:** REC focuses purely on real estate entities, Brick handles all technical equipment and connectivity
- **Comprehensive Coverage:** Brick provides more detailed ICT equipment, sensor, and parameter modeling
- **Semantic Consistency:** ICT equipment hierarchy benefits from Brick's measurement, control point, and telemetry infrastructure
- **No Equipment Overlap:** Eliminates all ICT/technical equipment duplication between REC and Brick ontologies
- **No Duplication:** Eliminates overlapping equipment definitions between ontologies

**Decision 7: Swedish Translation Completeness (DECIDED)**

**Choice:** Comprehensive systematic Swedish translations  **SELECTED**

**Scope Analysis:**

- **Total terms:** 335 labels in current ontology
- **Missing translations:** 297 terms (89%) were copying English to Swedish
- **Coverage needed:** All REC domain concepts for Swedish market accessibility

**Systematic Translation Organization:**

**Domain Categories** (284 comprehensive translations):

- **Core Concepts** (10): Entity, Space, Asset, Agent, Collection, etc.
- **Architecture & Spaces** (18): Buildings, floors, rooms, zones, elements
- **Room Types** (34): Specialized rooms (conference, server, bathroom, etc.)
- **Equipment** (12): Technical equipment, sensors, controllers, networking
- **HVAC Systems** (15): Ventilation, heating, cooling equipment
- **Furniture** (22): Office and residential furniture items
- **Properties** (26): Relationships and attributes
- **Technical Terms** (9): Identifiers, geometries, technical parameters

- **Events** (5): System events and alarms
- **Collections** (2): Equipment and furniture groupings
- **Assets** (2): Specialized asset types
- **Security** (2): Access control concepts

#### Translation Principles:

- **Swedish Real Estate Terms:** Use standard Swedish property/facility terminology
- **Technical Accuracy:** Maintain technical precision in Swedish context
- **Natural Language:** Idiomatic Swedish rather than literal translations
- **Domain Consistency:** Consistent terminology within each domain
- **Future Extensibility:** Organized structure supports easy additions

**Implementation:** Structured by domain categories for maintainability and comprehension by Swedish-speaking users across real estate, facility management, and technical domains.

**Business Impact:** Enables full Swedish localization of REC-based applications for Nordic markets.

### Decision 8: Property Exclusion Review (DECIDED)

**Choice:** Current exclusions are complete and sufficient ✅ **SELECTED**

#### Comprehensive Analysis Results:

- ✅ **Custom Properties/Tags:** Properly excluded (non-semantic DTDL extensions)
- ✅ **Observation Subclasses:** All 60+ observation types excluded per Decision 6
- ✅ **Substance Enumerations:** Architecture.isFedBy and related patterns excluded
- ✅ **No Additional Exclusions Needed:** Systematic review found no other problematic properties

**Conclusion:** Property exclusion strategy is optimal - excludes exactly what should be excluded without over-excluding valid REC semantic properties.

### Decision 9: Validation Message Implementation (DECIDED)

**Choice:** Comprehensive validation message generation ✅ **SELECTED**

#### Implementation Scope:

- **Generic Messages:** Template-based messages for cardinality and class constraints
- **Property-Specific:** Custom messages for configured properties (hasPoint, geometry, etc.)
- **Enumeration Messages:** Validation messages for SHACL enumeration constraints
- **Language Support:** English messages with configurable language support

#### Implementation Results:

#### Generated Validation Messages:

```
# Relationship constraints with cardinality
rec:acknowledgedByShape sh:message
    "Property acknowledgedBy must reference a valid instance of Agent. Can have at
most one acknowledgedBy"@en .

# Class constraints
rec:addressShape sh:message
    "Property address must reference a valid instance of PostalAddress"@en .
```

```
# Enumeration constraints
rec:poeTypeShape sh:message
    "PoE type must be 802.3af/at Type 1-4"@en .

rec:generationShape sh:message
    "WiFi generation must be WiFi4, WiFi5, WiFi6, WiFi6E, or WiFi7"@en .
```

#### Technical Implementation:

- **Translator Enhancement:** Added `addValidationMessage()` method to `configurable-translator.js`
- **Union Type Support:** Fixed handling of union ranges (e.g., "rec:Architecture | rec:Collection")
- **Configuration-Driven:** Uses Decision 1 validation message configuration
- **SHACL Compliance:** Generates proper `sh:message` statements (not comments)

#### Coverage Statistics:

- **Total SHACL shapes:** 138 shapes generated
- **Validation messages:** Generated for relationships with constraints and configured enumerations
- **Message types:** Class constraints, cardinality constraints, enumeration constraints

#### Benefits:

- **Developer Experience:** Clear validation failure messages during SHACL validation
- **Configuration-Driven:** Easy to customize messages per property
- **Multilingual Ready:** Architecture supports future Swedish message addition
- **Standards Compliant:** Uses proper SHACL `sh:message` semantics

### Decision 10: Property Characteristics Implementation (DECIDED)

**Choice:** Comprehensive property characteristics generation  **SELECTED**

#### Implementation Scope:

- **Essential Characteristics:** `owl:AsymmetricProperty` , `owl:IrreflexiveProperty` for safety-critical relationships
- **Configuration-Driven:** Uses Decision 2 property characteristics configuration
- **Selective Application:** Only applied to configured spatial and hierarchical properties
- **Future Extensible:** Supports all OWL property characteristics (transitive, functional, etc.)

#### Implementation Results:

##### Generated Property Characteristics:

```
# Spatial relationships with safety characteristics
rec:locatedIn
    a owl:AsymmetricProperty, owl:IrreflexiveProperty, owl:ObjectProperty ;
    rdfs:subPropertyOf rec:relationship .

rec:mountedOn
    a owl:AsymmetricProperty, owl:IrreflexiveProperty, owl:ObjectProperty ;
    rdfs:subPropertyOf rec:relationship .

# Hierarchical relationships preventing loops
rec:hasPart
```

```

    a owl:AsymmetricProperty, owl:IrreflexiveProperty, owl:ObjectProperty ;
    rdfs:subPropertyOf rec:relationship .

rec:isPartOf
    a owl:AsymmetricProperty, owl:IrreflexiveProperty, owl:ObjectProperty ;
    rdfs:subPropertyOf rec:relationship .

# Asset-Point relationships maintaining clean separation
rec:hasPoint
    a owl:AsymmetricProperty, owl:IrreflexiveProperty, owl:ObjectProperty ;
    rdfs:subPropertyOf rec:relationship .

rec:isLocationOf
    a owl:AsymmetricProperty, owl:IrreflexiveProperty, owl:ObjectProperty ;
    rdfs:subPropertyOf rec:relationship .

```

#### Technical Implementation:

- **Method Added:** `addPropertyCharacteristics()` in `configurable-translator.js`
- **Integration:** Called during relationship translation for configured properties
- **OWL Mapping:** Maps characteristic strings to proper OWL vocabulary terms
- **Logging:** Provides feedback on applied characteristics during translation

#### Applied Characteristics:

- **6 Properties Enhanced:** `hasPart`, `isPartOf`, `locatedIn`, `isLocationOf`, `hasPoint`, `mountedOn`
- **2 Characteristics Each:** `AsymmetricProperty` (prevents  $A \rightarrow B$ ,  $B \rightarrow A$ ) + `IrreflexiveProperty` (prevents  $A \rightarrow A$ )
- **Safety Benefits:** Prevents circular containment, spatial loops, and bidirectional hierarchies

**Brick Schema Alignment:** Matches Brick's approach where critical relationships use essential characteristics for consistency checking while keeping complexity manageable.

#### Benefits:

- **Logical Consistency:** OWL reasoners can detect modeling errors (circular hierarchies, spatial containment loops)
- **Data Quality:** Prevents invalid relationship patterns during data validation
- **Semantic Precision:** More accurately represents real-world constraints of spatial and hierarchical relationships
- **Future Ready:** Architecture supports adding transitive, functional properties when needed

#### DTDL Interface → OWL Class + SHACL NodeShape

```

// From DTDL
{
  "@id": "dtmi:org:w3id:rec:Space;1",
  "@type": "Interface",
  "displayName": {"en": "Space"}
}

// To OWL/SHACL
rec:Space a owl:Class, sh:NodeShape ;

```

```
rdfs:label "Space"@en, "Utrymme"@sv ;
rdfs:subClassOf rec:Entity .
```

### DTDL Relationship → OWL ObjectProperty + SHACL PropertyShape

```
// From DTDL
{
  "@type": "Relationship",
  "name": "geometry",
  "target": "dtmi:org:w3id:rec:Geometry;1",
  "maxMultiplicity": 1
}

// To OWL/SHACL (following universal relationship pattern)
rec:geometry a owl:ObjectProperty ;
  rdfs:label "geometry"@en, "geometri"@sv ;
  rdfs:subPropertyOf rec:relationship .

rec:geometryShape a sh:PropertyShape ;
  sh:path rec:geometry ;
  sh:class rec:Geometry ;
  sh:maxCount 1 .
```

### DTDL Component → Separate OWL Class + Relationship

```
// From DTDL
{
  "@type": "Component",
  "name": "area",
  "schema": "dtmi:org:w3id:rec:ArchitectureArea;1"
}

// To OWL/SHACL (component pattern)
rec:ArchitectureArea a owl:Class, sh:NodeShape ;
  rdfs:label "Architecture Area"@en, "Arkitektur Yta"@sv ;
  rdfs:subClassOf rec:Information .

rec:hasArea a owl:ObjectProperty ;
  rdfs:label "has area"@en, "har yta"@sv ;
  rdfs:subPropertyOf rec:relationship ;
  rdfs:domain rec:Architecture ;
  rdfs:range rec:ArchitectureArea .
```

### DTDL Enumeration → SHACL Constraint

```
// From DTDL
{
  "@type": "Property",
```

```

    "name": "poeType",
    "schema": {
      "@type": "Enum",
      "enumValues": [{"enumValue": "Type1"}, {"enumValue": "Type2"}]
    }
  }

// To SHACL (small enum strategy)
rec:EthernetPortShape sh:property [
  sh:path rec:poeType ;
  sh:in ( "Type1" "Type2" "Type3" "Type4" ) ;
  sh:datatype xsd:string ;
  sh:maxCount 1 ;
  sh:message "PoE type must be 802.3af/at Type 1-4"
] .

```

## DTDL Map → Intermediate Class

```

// From DTDL
{
  "name": "identifiers",
  "schema": {
    "@type": "Map",
    "mapKey": {"schema": "string"},
    "mapValue": {"schema": "string"}
  }
}

// To OWL/SHACL (identifier pattern from recommendations)
rec:Identifier a owl:Class ;
  rdfs:label "Identifier"@en, "Identifierare"@sv .

rec:hasIdentifier a owl:ObjectProperty ;
  rdfs:domain rec:Entity ;
  rdfs:range rec:Identifier .

rec:namespace a owl:DatatypeProperty ;
  rdfs:domain rec:Identifier ;
  rdfs:range xsd:string .

rec:value a owl:DatatypeProperty ;
  rdfs:domain rec:Identifier ;
  rdfs:range xsd:string .

```

## Namespace Strategy

- **REC Core:** <http://w3id.org/rec#>
- **Brick Integration:** <https://brickschema.org/schema/Brick#>
- **Standard Prefixes:** owl: , rdfs: , sh: , xsd:

## Integration with Brick Schema

REC maintains clean separation while enabling integration:

```
rec:Asset a owl:Class, sh:NodeShape ;
  rdfs:label "Asset"@en, "Tillgång"@sv .

rec:hasPoint a owl:ObjectProperty ;
  rdfs:label "has point"@en, "har punkt"@sv ;
  rdfs:domain rec:Asset ;
  rdfs:range brick:Point .
```

This allows REC assets to reference Brick measurement points while maintaining ontological independence.

## Remaining Implementation Fixes

### Priority Issues to Address

#### ✗ 1. Eliminate Observation Subclasses

```
# Current (incorrect): Multiple observation classes
rec:TemperatureObservation, rec:PressureObservation, rec:FlowObservation...

# Target (correct): Single observation class with Brick Point context
rec:ObservationEvent a owl:Class, sh:NodeShape ;
  rdfs:subClassOf rec:Event ;
  rdfs:label "Observation Event"@en, "Observationshändelse"@sv .

# Semantic context comes from Brick Point
ex:TempSensor a brick:Temperature_Sensor ;
  brick:hasQuantity brick:Temperature ;
  rec:lastKnownValue [
    a rec:ObservationEvent ; # Generic, not TemperatureObservation
    rec:value 22.5
  ] .
```

#### ✗ 2. Eliminate Architecture.isFedBy Substance Enumeration

```
# Current (incorrect): String enumeration duplicates Brick substances
rec:Architecture rec:isFedBy "ChilledWater" . # String literal

# Target (correct): Use Brick substance classes via equipment relationships
ex:Building rec:isLocationOf ex:ChillerEquipment .
ex:ChillerEquipment brick:hasPoint ex:ChilledWaterSensor .
ex:ChilledWaterSensor brick:hasSubstance brick:Chilled_Water .

# Query for building substances via equipment
SELECT ?building ?substance WHERE {
  ?building rec:isLocationOf ?equipment .
  ?equipment brick:hasPoint ?point .
```

```
    ?point brick:hasSubstance ?substance .  
}
```

### ❌ 3. Expand Swedish Translations

```
# Add comprehensive domain vocabulary  
"Equipment" → "Utrustning"  
"Sensor" → "Sensor"  
"Temperature" → "Temperatur"  
"Pressure" → "Tryck"  
"Flow" → "Flöde"  
"Building" → "Byggnad"  
"Floor" → "Våning"  
"Room" → "Rum"  
"HVAC" → "Ventilation"  
"Chiller" → "Kylmaskin"  
"Boiler" → "Panna"  
"Air Handler" → "Luftbehandlingsaggregat"  
"Fan" → "Fläkt"  
"Pump" → "Pump"  
"Valve" → "Ventil"  
"Damper" → "Spjäll"  
"Filter" → "Filter"  
"Coil" → "Spole"  
"Heat Exchanger" → "Värmeväxlare"
```

## Future Extensions

1. **QUDT Integration:** For quantities and units in observation events
2. **SKOS Vocabularies:** For large enumerations and taxonomies
3. **Time Ontology:** For temporal aspects of asset lifecycle
4. **Geographic Standards:** Integration with spatial reference systems

## Configuration-Driven Translation

### Domain-Specific Decision Points

The translator will support configurable decisions for:

1. **Missing Relationship Ranges:** User-defined mappings for relationships without targets
2. **Enumeration Strategies:** Per-enum choice of SHACL vs SKOS vs OWL individuals
3. **Component Handling:** Custom rules for DTDL Component translation
4. **Swedish Translation Overrides:** Domain-specific translation preferences
5. **Property Exclusions:** Configurable exclusion rules beyond defaults

### Configuration Structure

```
{  
  "missingRanges": {  
    "Agent.owns": "rec:Asset",  
    "Portfolio.includes": "rec:RealEstate",
```

```
    "Document.url": "xsd:anyURI"
  },
  "enumerationStrategies": {
    "ServiceObject.severity": "owl_individuals",
    "WirelessAccessPoint.generation": "skos_concepts"
  },
  "swedishTranslations": {
    "Equipment": "Utrustning",
    "Sensor": "Sensor"
  },
  "excludeProperties": ["customProperties", "customTags"],
  "excludeObservationSubclasses": true,
  "excludeSubstanceEnums": true
}
```

---

*This document establishes the foundational principles for creating a robust, multilingual, and semantically rich REC ontology that complements rather than duplicates Brick Schema capabilities.*